

TSKS37 Lab 3

Data Compression and Wireless Transmission

Fall 2026

(Version of This Document: August 27, 2026)

Examination

This exercise is a set of tasks described further down in the document. We expect you to present the required figure(s). The lab assistant may also ask for an explanation of the written code.

Submission

After passing the labs, the code of Lab 2 and Lab 3 should be uploaded to the Lisam study room under Submissions. The course director will send the code to Ouriginal. The code from these two labs should be saved in one text document called `coursecode_year_studentid1_studentid2.txt` (e.g., `TSKS37_2024_adaer110_evaer120.txt`). In case you are using Jupyter lab you can export your code through: `file > Export Notebook as > Executable Script`.

1 Introduction

In this exercise, you will practice using NumPy to simulate a communication system from start to finish and evaluate its performance. The goal of this lab is to model the process of transmitting data efficiently. You will begin by compressing a data source to reduce transmission load and encode the data by adding parity bits to improve communication reliability. Then, you will check the communication medium for active users ready to receive the data. Finally, you will transmit the data to the user and assess the performance of the transmission by calculating the bit error rate (BER) and symbol error rate (SER).

To pass this exercise you need to show and demonstrate the script and/or functions that you have created to solve the main tasks. Keep a balance between comments as well as good variable/function names in the code, well written code is easy to read and speaks for itself. Ensure that the plots are self-explaining.

1.1 Main Tasks

The lab is structured into five main parts, which are discussed in more detail below.

1. **Data compression (Huffman coding):** To make the communication system efficient, data compression can be employed to reduce the number of bits transmitted. The data for transmission in this lab consist of a long sequence of English text. We will perform data compression using Huffman coding. Note that compression can also be applied to other types of information, such as images and audio files.
2. **Encoding with Hamming code:** Hamming code is an error-correcting code that adds parity bits to the source data to detect double-bits or correct single-bit errors during transmission, thereby improving communication reliability.
3. **Identify the active user:** In a scenario where multiple users are present within a communication network, determining which ones are active is important. One approach could be to ask each user individually if they want to proceed with whatever communication protocol is in place, but this could be seen as a waste of resources. Instead, we can ask multiple users at once and then interpolate which users are active, from what we receive from each group.
4. **Transmit to active user:** When the active user is identified, we need to transmit the data to the user. The encoded bit sequence is modulated and transmitted over the additive white Gaussian noise (AWGN) channel.
5. **Evaluate BER and SER with Monte-Carlo simulations** Finally, to evaluate the quality of our communication channel, we simulate multiple trials and average the number of bits that are incorrectly decoded.

1.2 Libraries

We will use the following Python libraries:

- Numpy
- Matplotlib

There are many other libraries we could use that would make the lab easier (or more difficult). We encourage you to try those libraries on your own.

2 Theory

In this section, we introduce some of the theory used in the lab.

2.1 Data compression (Huffman coding)

Huffman coding is a lossless data compression method that assigns shorter bit codes to more frequent symbols and longer codes to less frequent ones using Huffman tree. It is a prefix-code (also called prefix-free code in some literature) which means that no codeword is a prefix to another codeword, hence prefix-code is uniquely decodable code. We first denote *alphabet* $\mathcal{A}$ as the set of *symbols*, in this case the English characters. For instance, the alphabet $\mathcal{A}$ for English characters (including other symbols, such as punctuation) can be represented as

$$\mathcal{A} \in \{A, a, B, b, C, c, \dots, (, \&, ?\}.$$

A codeword is a binary sequence that represents an original symbol. For instance, a set of codeword for English characters can be represented as

$$C = \{a = 11, b = 1010, c = 1011, d = 100, \dots\}.$$

Encoding is the process of transforming the original data (sequence of English text in this case) to the compressed form. Therefore, in the example, 'cab' is encoded into 101111010. Likewise, If there is no error during transmission, the decoding is done by just replacing the received bit sequence, for instance, 101011100 is 'bad'. Since this is a prefix code, the decoded bit sequence is unique. The main task in Huffman coding is to construct a binary tree (also called the Huffman tree) for the codewords. To construct the Huffman tree (a bottom up method):

- **Step 1.** Derive the symbol distribution based on the given file. For example, given the count for each symbol as [a:10], [b:2], [c:3], [d:4], [e:17]
- **Step 2.** Sort the nodes by frequency from low to high:
[b:2], [c:3], [d:4], [a:10], [e:17]
- **Step 3.** (Iteration 1) Merge the smallest nodes, in this case **b** (2) and **c** (3). The sum of the new nodes is 5.
The updated pool (sorted) is: [d:4], [(bc):5], [a:10], [e:17].
- **Repeat step 3 until reach root node:**
(Iteration 2) Merge smallest nodes **d** (4) and **(bc)** (5). The sum of the new nodes is 9. The updated pool (sorted) is: [(dbc):9], [a:10], [e:17].
(Iteration 3) Merge smallest nodes **(dbc)** (9) and **a** (10). The sum of the new nodes is 19. The updated pool (sorted) is: [e:17], [(dbca):19].
- **Final Merge:** (Iteration 4) Merge **d** (17) and **(dbca)** (19). Root node sum is 36.

The Huffman tree diagram for the above example is shown in Figure 1 and the corresponding codebook table is given in Table 1.

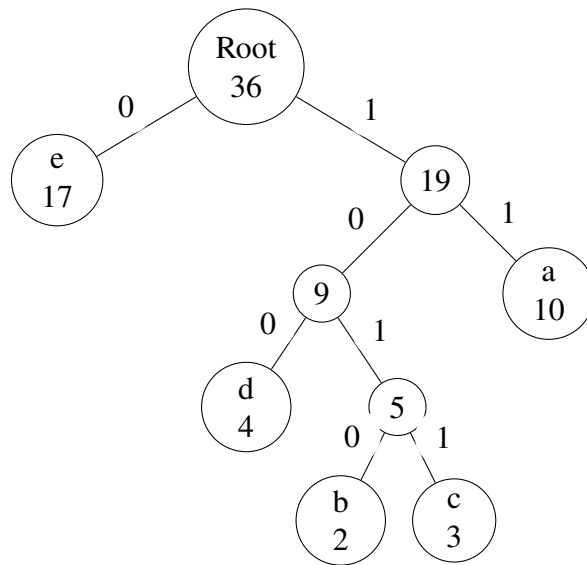


Figure 1: Huffman tree (Left branches represent '0', right branches represent '1').

Table 1: The codebook corresponding Huffman tree in Figure 1

Char	Freq	Path (L=0, R=1)	Huffman Code
e	17	Left	0
a	10	Right → Right	11
d	4	Right → Left → Left	100
b	2	Right → Left → Right → Left	1010
c	3	Right → Left → Right → Right	1011

2.2 Channel Coding (Hamming Code)

In digital communication, errors are commonly introduced into the information sequence. Due to noise, interference, or hardware issues, bits may flip from 0 to 1 (vice versa) and hence cause errors. Without detection/correction, the receiver may obtain incorrect information from the data sequence with errors. Hamming codes are a typical family of error-correcting codes that help to detect/correct errors by adding a small amount of designed redundancy to the bit sequence (parity bits).

Assuming that the original message is grouped into blocks of k information bits, the Hamming code is a linear block code that encodes the information bits by adding r parity bits. The resulting encoded message is of $n = k + r$ bits. The redundancy is the “cost” of robustness.

The most important feature of a Hamming code is that it can often locate the error position and then correct it. At the receiver, we compute a vector called the *syndrome*:

$$s = H \cdot y^T,$$

where H is the parity-check matrix of the Hamming code, and y is received vector having n bits. If $s = 0$, there is no error in the encoded message; If $s \neq 0$, it becomes a “signature” indicating that there are errors in the message. If there is a single-bit error, the syndrome can locate the error position according to the coding rule. Once the position is known, correction is simple: flip the bit. If there is a double-bit error, the syndrome can only detect that an error has occurred.

Now, let’s start to design a (7, 4) Hamming code, which means the code can encode a 4-bit information into a 7-bit message with 3-bit parity ($k = 4$, $r = 3$, $n = 7$):

- **Construct the parity-check matrix H .** H has size $r \times n$. The most common method is to assign each column of H a unique non-zero r -bit binary vector. As a result, if the i -th bit is wrong, the syndrome becomes exactly the i -th column of H . An systematic way of constructing H is to have a generator matrix fulfilling the form $H = [P^T \ I_r]$, where P is parity submatrix. An example of matrix H is

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}.$$

In this way, if the syndrome is $[1 \ 0 \ 1]^T$, an error happens to the 2-nd bit of the encoded message as it matches the 2rd column in H . If the syndrome is $[0 \ 0 \ 0]^T$, there is no error in the encoded message.

- **Construct the generator matrix G .** G has size $k \times n$. An systematic way of constructing G is to have a generator matrix fulfilling the form $G = [I_k \ P]$, where I_k is identity matrix with dimension k . The generator matrix G must satisfy $G \cdot H^T = 0$, where H is the parity-check matrix. An example of matrix G is

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

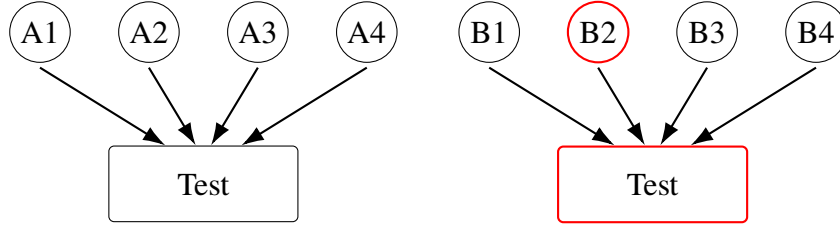


Figure 2: Group testing example

- **Encode.** For the original message $\mathbf{u}$ with the size $1 \times k$. The encoded message is generated by: $\mathbf{c} = \mathbf{u} \cdot \mathbf{G}$. For example, for $\mathbf{u} = [1 \ 0 \ 1 \ 1]$, the encoded message from the given matrix $\mathbf{G}$ is $\mathbf{c} = [1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0]$, where first 4-bits are the original message and the latter 3-bits are the parity bits.
- **Transmission.** By transmission, errors can happen to $\mathbf{c}$. The received message is termed $\mathbf{r}$.
- **Decode.** Compute $\mathbf{s} = \mathbf{H} \cdot \mathbf{r}^\top$ and correct the error. For example, if the received message is $\mathbf{r} = [1 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0]$ and the syndrome is $\mathbf{s} = \mathbf{H} \cdot \mathbf{r}^\top = [0 \ 1 \ 1]^\top$. Hence, the error occurs at bit 3 as the syndrome match with 3rd column. The original message is $\mathbf{c} = [1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0]$, where bit 3 is flipped.

2.3 Group Testing

When analyzing a set of objects, individual testing provides complete information but scales poorly as the set grows. To reduce the number of required tests, *group testing* can be used, in which tests are performed on subsets of objects rather than on each object separately.

A common example arises in quality control: if defects are rare, testing every item is inefficient. By testing items in groups, the number of tests can be significantly reduced, and group sizes can be chosen to optimize resource use.

Figure 2 illustrates this with eight items. Testing them individually requires eight tests. If we instead divide them into groups A and B, we may determine the status of all items with as few as two tests when no defects are present. In the illustrated case, only item B2 is defective, so group B tests positive and we then test items B1–B4 individually. This strategy requires only five tests in total, fewer than the eight needed for full individual testing.

2.4 Data transmission

Modulation: The modulation is needed to transmit the bit sequence over carrier signal. For simplicity, we consider binary phase-shift keying (BPSK) modulation. BPSK is a simple digital modulation technique that transmits data by shifting the phase of a carrier by 180 degrees to represent bit 0 and bit 1. We use unit power symbol for BPSK, hence bit 1 is modulated as '+1'

and bit 0 is modulated as '-1'.

Channel: The modulated signal is transmitted over the AWGN channel as

$$\mathbf{y} = \mathbf{x} + \mathbf{n}, \quad (1)$$

where $\mathbf{y}$ is the received signal at the receiver, $\mathbf{x}$ is the modulated signal, and $\mathbf{n} \sim \mathcal{N}(0, \text{SNR}^{-1})$ is the additive white Gaussian noise, and SNR is the signal-to-noise ratio.

ML Detector and decoding: At the receiver, the maximum likelihood (ML) detector for BPSK to demodulate bit sequence is

$$y_m \underset{0}{\overset{1}{\geq}} 0, \quad (2)$$

where y_m is the m th symbol of the received signal $\mathbf{y}$. The received bit sequence is decoded (Hamming) with correction if there is at most one-bit error. Finally, the original text is recovered with Huffman decompression.

2.5 Monte-Carlo Simulation

To accurately determine the performance of the communication system with the BER and SER, it is necessary to perform a Monte Carlo (MC) simulation. This method involves repeating an experiment multiple times and calculating the average result to account for the inherent randomness present in the experiment. In this scenario, our goal is to repeat the entire transmission process and calculate the BER and SER for each instance. Note that for a BER of around 10^{-5} , at least approximately 10^5 trials is required for MC simulation to observe single error.

3 Task

The overall objective of this lab is to design a communication system that transmits data over a wireless channel from the access point to the active user (Figure 3). All tasks should be done in the following order and are described below.

3.1 Compress data (Huffman coding)

1. Load `FILE:training-text.txt` into a string.
2. Create a python dictionary with each symbol as the key, and the number of times it occurs in the data as the value. For example `{ 'a':45, 'b':13, ... }`.
3. Follow section 2.1 to create a binary tree where each path to a leaf gives the bit sequence for a coded symbol.
4. Create a mapping table to store symbols and codewords in a key-value table (i.e., a Python dictionary) using the tree from previous step. This will be used for both compression and decompression.

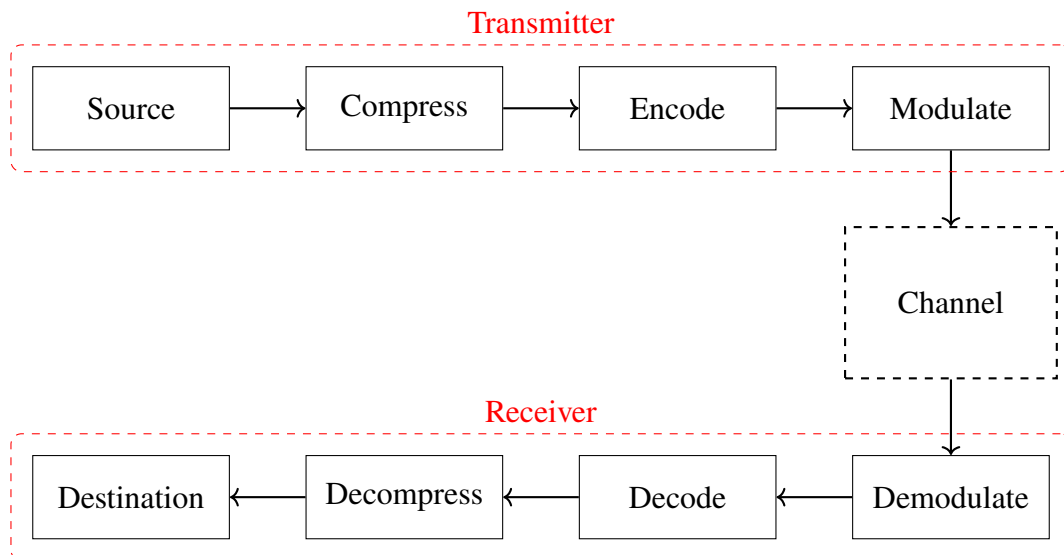


Figure 3: Flowchart of a typical communication channel

5. *Compress* (encode) `FILE:romeo-and-juliet.txt` the first 200 characters `[:200]` and use them as data by changing each symbol to its corresponding bit sequence. Also, *decompress* (decode) and verify that your encoder and decoder works by checking the original text is equal ('==') to the decoded text.
6. Calculate the compression ratio defined as

$$\text{Compression Ratio} = \frac{\text{Uncompressed Size}}{\text{Compressed Size}}, \quad (3)$$

where the uncompressed size is the number of symbols in the text times 8 (assuming ASCII), and the compressed size is the number of bits in the compressed version.

Hints:

- Don't forget spaces and special characters in your codebook!
- The binary tree might look different depending on implementation, so there is no *correct* tree. However, the compression ratio should be the same for all implementations.

3.2 Encoding

1. Given the bit sequence from Task 3.1, add zero padding if the length of your bit sequence is not divisible by 4 i.e., if $\text{len}(\text{bits}) \bmod 4 \neq 0$. The bit sequence should be divisible by 4 and store the padding value.
2. *Encode* the message by calculate the product between your padded bit sequence and generator matrix G as described in Section 2.2, $\mathbf{c} = \mathbf{u} \cdot G$. Note that the encoding is done 4 bits at a time by segmenting the bit sequence. Finally, the encoded bit sequence is created by concatenating each encoded segment.

3. What is the length of the bit sequence before and after Hamming encoding? What is the ratio of information bits (before encoding) to total bits (after encoding)? Does it match with k/n ?

Hints:

- Carefully handle data types, i.e., `str` and `array of int`.

3.3 Identify the active user

1. In the case of 6 total users, explain how you can identify the active user with at most 4 active checks, given that there is exactly one active user. Note that implementation is not required for this task; only an explanation of how group testing can be applied in this scenario is needed.

3.4 Modulate and transmit to active user

1. *Modulate* your encoded bit sequence from Task 3.2 by turning 1's into +1's, and 0's into -1's.
2. Add white Gaussian noise to your modulated symbols with SNR between -2 and 10 dB. You may try using a single SNR value to verify the implementation.

Hints:

- SNR in dB can be converted to linear scale with $\text{SNR} = 10^{\text{SNR (dB)}/10}$.
- `numpy.random.randn()` gives Gaussian random realizations of mean 0 and variance 1.

3.5 Demodulate, decode and decompress

- *Demodulate* the received signal using the ML-decoder explained in 2.4.
- *Decode* your demodulated bit sequence from previous step with parity-check matrix as explained in subsection 2.2, $s = H \cdot r^T$. Note that the decoding is done 7 bits at a time by segmenting the bit sequence. Finally, the decoded bit sequence is created by concatenating each decoded segment.
- Correct each received segment (4 bits) if the syndrome is not the zero vector during decoding in the previous step as described in subsection 2.2.
- Remove padding bits (if any) from 3.2 and decompress the data using the Huffman codebook from the first task.
- Compare the decompressed text with original text and find the SER as $\text{SER} = \frac{\text{\#Error symbols}}{\text{\#Total symbols}}$. Print the decompressed text and check if it is reasonable compared to the SER value.

- Compare the decoded bit sequence (after correction) with compressed bit sequence and find the BER as $BER = \frac{\text{\#Error bits}}{\text{\#Total bits}}$.
- Redo the task, but this time skip the Hamming encoding/decoding step and calculate the uncoded BER and uncoded SER. How does the performance of uncoded case look like compared to the coded case?

3.6 Evaluate performance with MC

- Run Monte Carlo simulations (2.5) for SNR values between -2 and 10 dB with 2 dB spacing, and plot the BER for both coded and uncoded cases in the same figure, with the x-axis representing SNR (in dB) and the y-axis representing BER on a logarithmic scale.
- Similarly with SER, plot the SER for both coded and uncoded cases in the same figure, with the x-axis representing SNR (in dB) and the y-axis representing SER on a logarithmic scale.

Hints:

- `plt.semilogy()` can be used to plot a log scale on the y-axis without manually converting the BER.
- For a BER of around 10^{-5} , at least approximately 10^5 trials is required for MC simulation to observe single error.